



Dynamic Programming

Presenter: Dr. Ha Viet Uyen Synh.



Introduction

What is a dynamic programming?

A DP is an algorithmic technique which is usually based on a **recurrent formula** and one (or some) starting **states**. A **sub-solution** of the problem is constructed from previously found ones.

What does a "state" stand for?

It's a way to describe a situation, a sub-solution for the problem.

How can we find it?

To be able to use DP, the original problem must have:

1. Optimal sub-structure property

optimal solution to the problem contains within it optimal solutions to sub-problems

2. Overlapping sub-problems property

we accidentally recalculate the same problem twice or more.



Characteristics of Dynamic Programming

Can we break down the problem into smaller sub-problems?

Are there any overlapping sub-problems?

If yes, can we solve the smaller sub-problems optimally, and then use them to construct an optimal solution to solve the main problem?

While solving the problem, does the decision on the current step affect the outcome and decision of subsequent steps?

“There are two key attributes that a problem must have in order for dynamic programming to be applicable: optimal substructure and overlapping sub-problems”



Types of Dynamic Programming Questions

The first type of dynamic programming question, which is also the frequently encountered kind, is to find an optimal solution for a given problem. Examples include finding the maximum profit, the minimum cost, the shortest path, or the longest common subsequence.

The second type of dynamic programming question is to find the possibility of achieving a certain outcome, the possibility of reaching a certain point, or the possibility of completing a task within the specified conditions.

The third type is usually referred to as a dynamic programming counting problem (or combinatorial problem), such as finding the number of ways to perform or accomplish a task.

Note that all three types of dynamic programming questions that we discussed could also appear as two-dimensional problems in the form of a matrix. More complex problems could involve multiple dimensions.



Approaches to Implement Dynamic Programming

Generally, the top-down approach starts by looking at the big picture. From the high level, it gradually drills down into the solution for smaller sub-problems, and finally, the base case for constructing the solution.

As opposed to the top-down approach, the bottom-up approach starts small by first looking at the base case, and from there step-by-step building up the solution to bigger sub-problems, before solving the entire problem as a whole.



Top-Down Approach

The implementation of a top-down approach uses recursion with *memorization*.

Memorization is the technique of storing the result of calculations so that they can be retrieved and used directly when the program requires them again. This is especially useful for overlapping sub-problems as it helps to avoid performing the same calculation twice, thus improving efficiency and saving compute time. Hashmap or array are commonly used for memorization.

This approach will make recursive function calls as per the recurrence relation that we define, as long as the current state is not the base case.



Bottom-Up Approach

To implement the bottom-up approach, we would need to iterate over all the states of the problem in a specific order, starting from the base case.

By doing so, we build up the solution step by step from small to big, such that the answer for the current step can easily be computed using the available sub-problem solutions computed from the earlier steps. This process keeps going until we build up the entire solution.

The bottom-up approach is also known as the *tabulation method*. Since this approach would go through each step in a specific order and perform computation, it is easy to tabulate the results in an array or list, where they can be conveniently retrieved by the relevant index for use in subsequent steps.

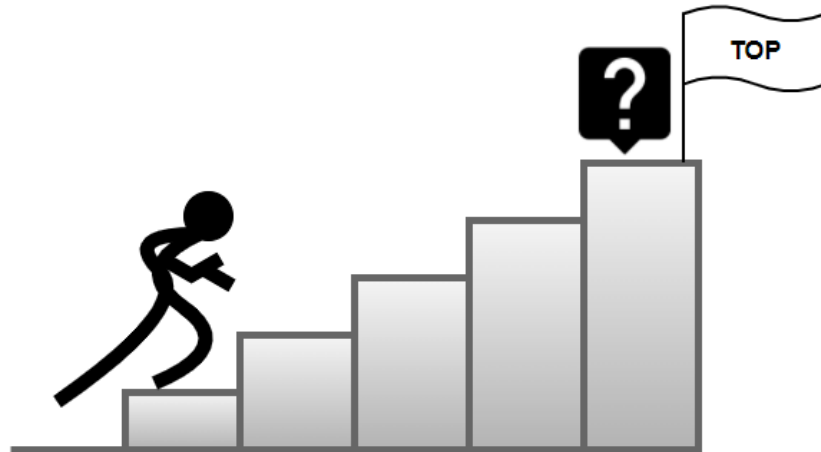


EXAMPLES

1. Climbing Stairs

Problem: we are given a staircase that would take us n steps to reach the top. Each time, we are only allowed to climb 1 step or 2 steps.

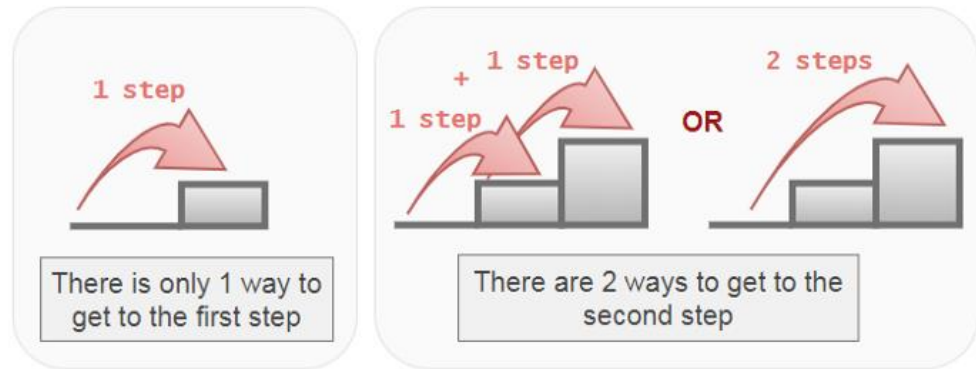
How many different ways can we climb to the top?





If we were to get to the first step, how many ways can we do that? Of course, there is only 1 way, and that is to climb 1 step.

How about getting to the second step? There are 2 ways to accomplish this. We can either climb 1 step twice, or we can climb 2 steps once.

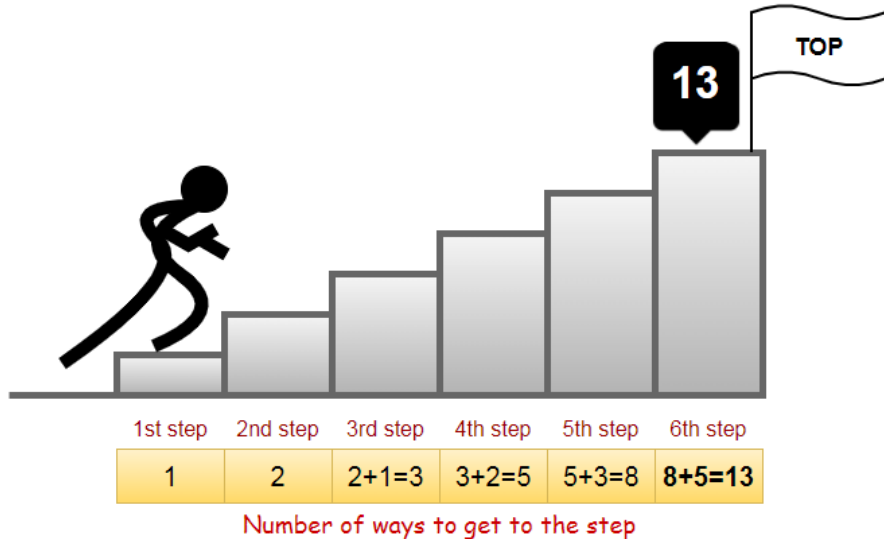


For the third step, there are just 3 possible ways to reach there. They are:
climb 1 step for 3 times, or
climb 1 step followed by 2 steps, or
climb 2 steps followed by 1 step

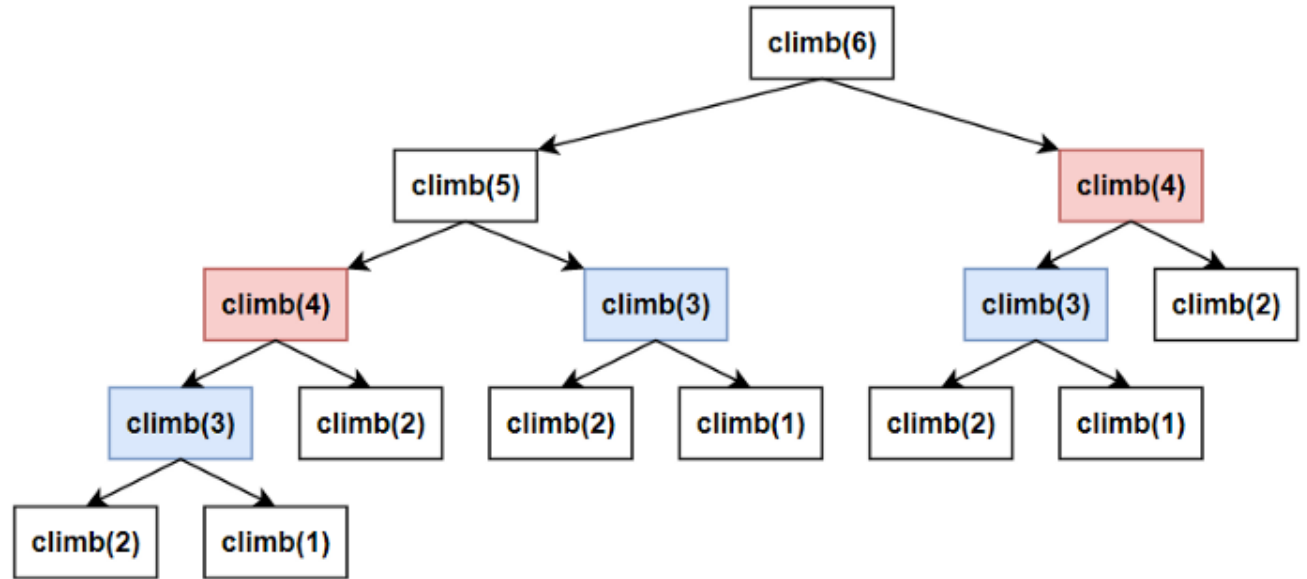


Assuming $\text{climb}(n)$ is the function that returns the number of ways to reach step n , then

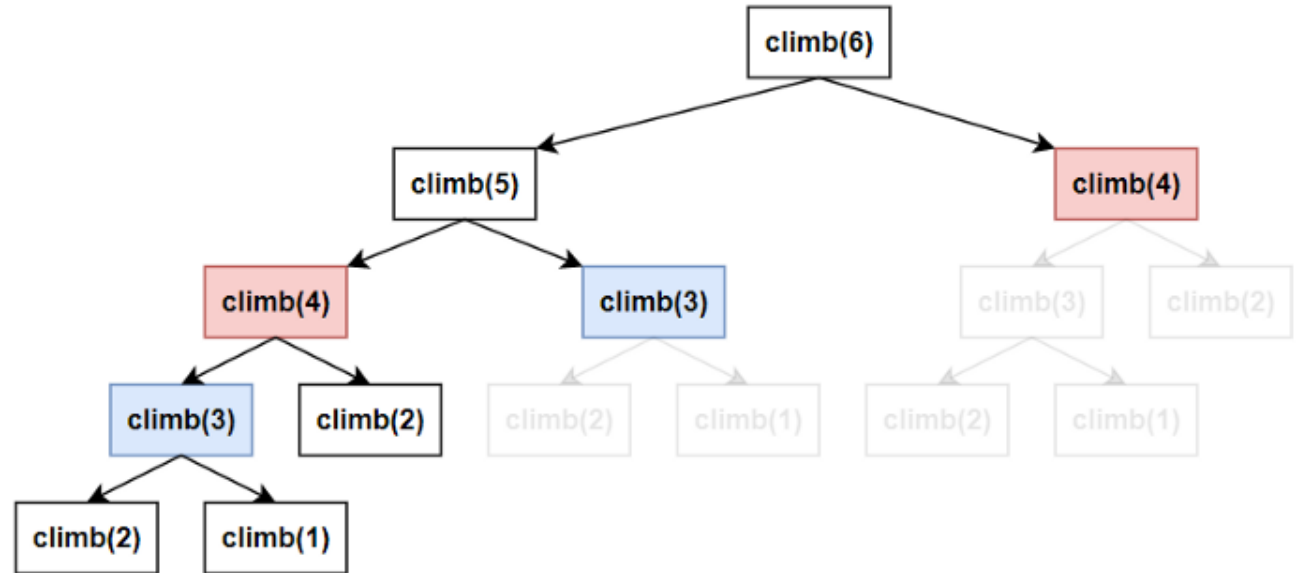
$$\text{climb}(n) = \text{climb}(n-1) + \text{climb}(n-2)$$



Overlapping sub-problems without memorization



Overlapping sub-problems with memorization





Top-down approach

```
result = {}          # Using hashmap for memorization.
```

```
def climb(n):
```

```
    # Base case
```

```
    if n==1: return 1    # Number of ways to get to first step is 1.
```

```
    if n==2: return 2    # Number of ways to get to second step is 2.
```

```
    # If n>2 and is not found in hashmap, perform calculation and store the result
```

```
    # Else, skip computation and retrieve the result from hashmap.
```

```
    if n not in result:
```

```
        result[n] = climb(n-1) + climb(n-2) # Recurrence relation
```

```
    return result[n]
```

```
# === Test case: =====
```

```
climb(6)
```



Bottom-up Approach

```
def climbStairs(n):
```

```
    if n==1: return 1
```

```
    if n==2: return 2
```

```
    # Base case
```

```
    case1 = 1
```

```
    # Number of ways to get to first step is 1.
```

```
    case2 = 2
```

```
    # Number of ways to get to second step is 2.
```

```
    # If the staircase has more than 2 steps (i.e. n>2), iterate from the third step onwards.
```

```
    for _ in range(2, n):
```

```
        result = case1 + case2 # Compute the ways to get to current step.
```

```
        case1 = case2 # Replace case1 with case2.
```

```
        case2 = result # Assign case2 with the current computation.
```

```
    return result
```

```
    # === Test case: =====
```

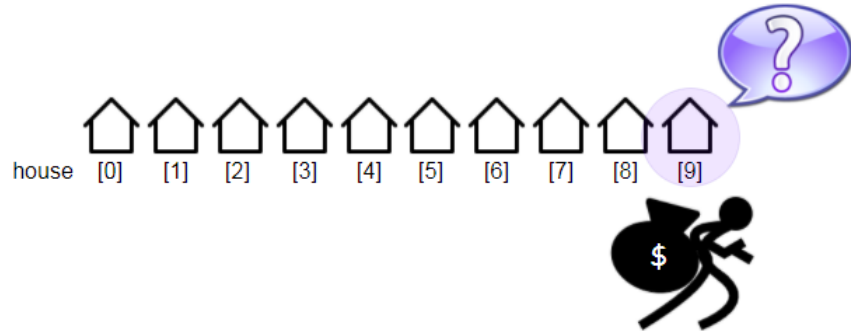
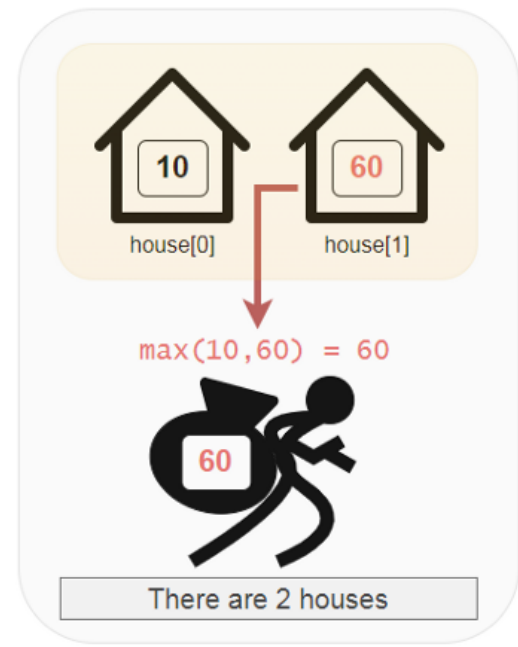
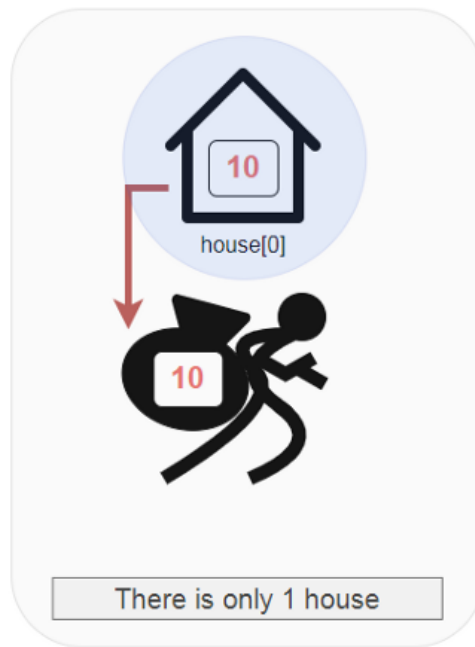
```
    climbStairs(6)
```

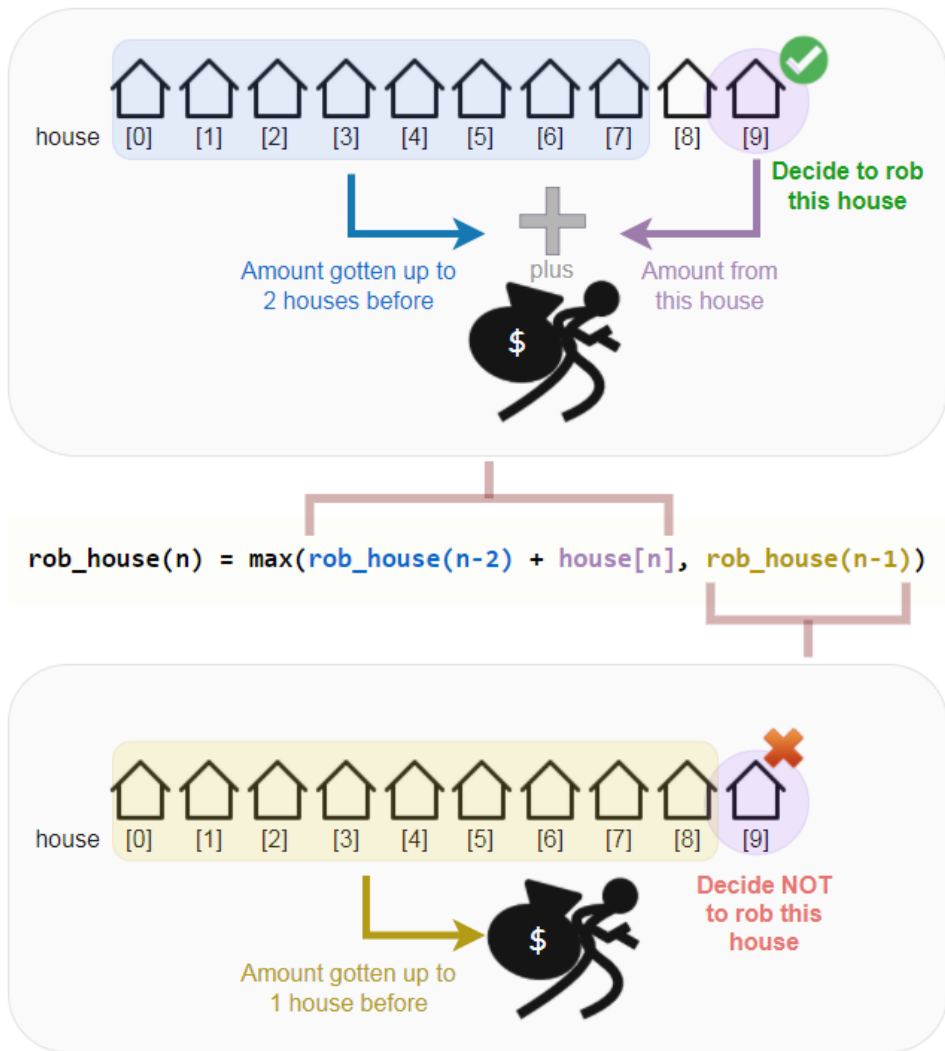
2. House Robber

Problem: We're given an array of integers, representing the amount of money in each house that a robber can rob. The houses are lined up on a street. There is one constraint though, that the robber cannot rob adjacent houses. For example, if the houses are indexed from 0 to 4, the robber can rob houses 0,2,4, or houses 1,3.

What is the maximum amount of money that the robber can rob from the houses?









Top-down approach

```
result = {}
```

```
# This is the rob_house() function that would be called recursively to solve sub-problems.
```

```
def rob_house(n):
```

```
    # Base case
```

```
    if n == 0:
```

```
        return house[0]
```

```
    elif n == 1:
```

```
        return max(house[0], house[1])
```

```
# Retrieve result from hashmap if exists, else calculate and store it in hashmap.
```

```
    if n not in result:
```

```
        result[n] = max(rob_house(n-2) + house[n], rob_house(n-1)) # Recurrence relation
```

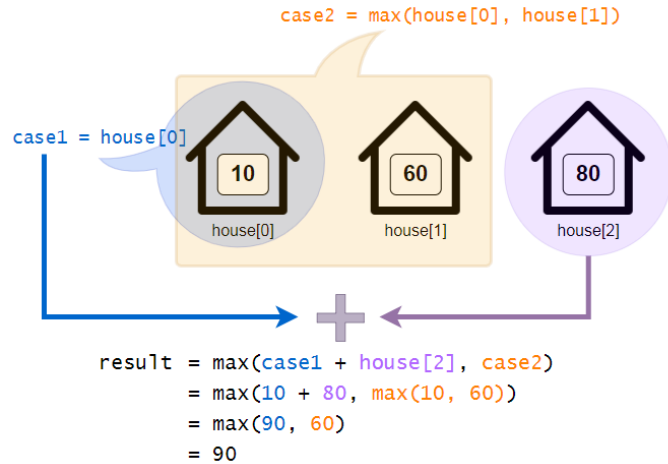
```
    return result[n]
```

```
# === Test case: =====
```

```
house = [10, 60, 80, 20, 8]
```

```
rob_house(len(house)-1)
```

Bottom-up Approach



```
def rob(house):
```

```
    if len(house) == 1:
```

```
        return house[0]
```

```
    elif len(house) == 2:
```

```
        return max(house[0], house[1])
```

```
    # Base cases
```

```
    case1 = house[0]
```

```
    case2 = max(house[0], house[1])
```

```
    # If there are more than 2 houses, iterate from house[2] onwards.
```

```
    for n in range(2, len(house)):
```

```
        result = max(case1 + house[n], case2) # Computation at current house
```

```
        case1 = case2      # Replace case1 with case2.
```

```
        case2 = result     # Assign case2 with the current computation.
```

```
    return result
```

```
# === Test case:=====
```

```
house = [10, 60, 80, 20, 8]
```

```
rob(house)
```

4. Cherry Pickup

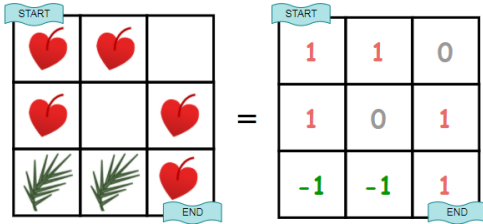
Problem: There is a $n \times n$ grid where the value in each cell can either be 1 (there's a cherry in the cell), 0 (the cell is empty), or -1 (there's a thorn, an obstacle that blocks the way).

Starting at cell (0,0), we are supposed to move to the destination, which is located at $(n-1, n-1)$. If there are cherries along the way, we pick them up. If we pick up a cherry from a cell, the cell becomes an empty cell, 0. We are only allowed to move down or right, and if there's a thorn, we will not be able to pass through.

After reaching the destination at $(n-1, n-1)$, we have to do a return trip to the starting cell (0,0). For the return trip, we are only allowed to move up or left, picking up remaining cherries, if any.

If we can't find a way to reach the destination at $(n-1, n-1)$, then we can't collect any cherries.

We are asked to return the maximum number of cherries we can collect.



Assuming person1 is at position (r1,c1) and person2 is at position (r2,c2) after a certain number of moves, we have $r1 + c1$ equal to $r2 + c2$.

This means we can use $r1$, $c1$, $r2$, $c2$ as state variables in our problem.

$$r1 + c1 = r2 + c2$$

Derive $r2$ as:

$$r2 = r1 + c1 - c2$$

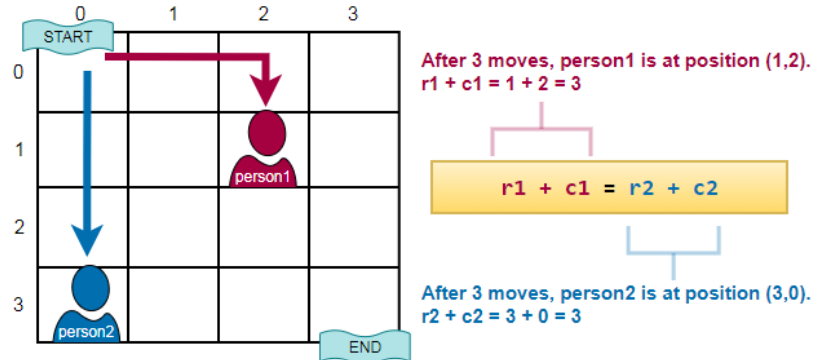
Where

$r1$ is the row index of person1

$c1$ is the column index of person1

$r2$ is the row index of person2

$c2$ is the column index of person2



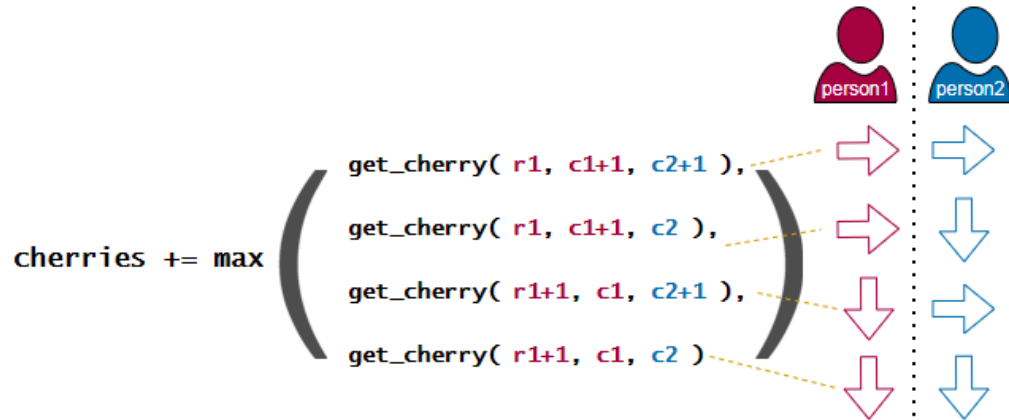
Since we have two persons moving simultaneously, there are four possibilities for each move, as depicted below:

	 person1	 person2
$r1, c1+1, c2+1$	person1 going right 	person2 going right 
$r1, c1+1, c2$	person1 going right 	person2 going down 
$r1+1, c1, c2+1$	person1 going right 	person2 going down 
$r1+1, c1, c2$	person1 going right 	person2 going down 

Presuming person1 is at position (r1,c1) and person2 is at position (r2,c2), and there are no thorns at both locations, we first add the value from (r1,c1) to cherries. Because there is no thorn, the value is either 0 or 1.

Next, to avoid double-counting, we would add the value from (r2,c2) to cherries only if person2 is at a different location.

So far, what we've done is accumulate cherries on the current location of person1 and person2. We would need to do the same accumulation for subsequent locations too.





Top-down approach

```
import functools
def cherryPickup(grid):
    last_index = len(grid)-1
    # Function will return max number of cherries picked, or -inf if there's no valid path.
    def get_cherry(r1, c1, c2):
        r2 = r1 + c1 - c2          # Derive r2 from r1, c1, c2.
        # Exception cases, return -inf:
        if (r1>last_index or c1>last_index or      # person1 is out of bound
            r2>last_index or c2>last_index or      # person2 is out of bound
            grid[r1][c1] == -1 or                  # person1 is on a cell with thorn
            grid[r2][c2] == -1):                  # person2 is on a cell with thorn
            return float('-inf')
        # Base case:
        elif r1==last_index and c1==last_index:    # If the last cell is reached,
            return grid[r1][c1]                    # return the value of the last cell.
        cherries = grid[r1][c1] # Location of person1, the value is either 0 or 1.
```



```
if c1 != c2: # If person2 is at a different location, add that cell value to the cherries count.
    cherries += grid[r2][c2]
# Next move have 4 options, find the max value and add to cherries count.
cherries += max(get_cherry(r1, c1+1, c2+1),    # person1 going right, person2 going right
                get_cherry(r1, c1+1, c2),      # person1 going right, person2 going down
                get_cherry(r1+1, c1, c2+1),    # person1 going down, person2 going right
                get_cherry(r1+1, c1, c2))      # person1 going down, person2 going down
return cherries
get_cherry.cache_clear()
return max(0, get_cherry(0, 0, 0))    # Start from cell (0,0).

# Test case:
cherryPickup([[1,1,0],[1,0,1],[-1,-1,1]])
```

A decorative image on the left side of the slide showing a stack of smooth, dark grey stones on a calm body of water. The stones are stacked horizontally, with the top stone being the most prominent. The water is a light blue-grey color, and the sky in the background is a pale, hazy blue.

5. Best Time to Buy and Sell Stock IV

Problem: We're given an integer k , and a list of prices (also in integers), where $\text{prices}[i]$ is the price of a stock on day i .

We're required to find the maximum profit that we can make by buying and selling the stock based on the given prices, with at most k transactions.

We can consider a transaction as performing a buy-and-sell action each on different days, where we may earn a profit after selling. Profit is defined as the value of sell price minus buy price. We can only perform one action (either a buy or sell action) on a particular day, and we have to sell the stock before buying again.

We would use the following sample input to walk through this problem:

$k=2$, $\text{prices}=[8, 15, 5, 70, 80, 100]$



Framework

2. Profit Table
3. Find Patterns
4. Solution Structure
5. Complexity Analysis
6. Look For Ways to Improve
 - (i) Make It Simple
 - (ii) Check for Redundancy
 - (iii) Determine the Scope of Dependency
7. Solution

Profit Table

prices =	8	15	5	70	80	100
profit table	day 0	day 1	day 2	day 3	day 4	day 5
txn 0	0	0	0	0	0	0
txn 1	0					
txn 2	0					

The values in the first row are all zeros.

We can't make any profit when there is no transaction.

The values in the first column are all zeros.
We can't complete a transaction and make any profit when there is only 1 price.

We need to have at least 2 prices to complete a buy-and-sell transaction.



Profits with 1 Transaction

prices = 8 $\leftrightarrow$ 15 5 70 80 100

profit table	day 0	day 1	day 2	day 3	day 4	day 5
txn 0	0	0	0	0	0	0
txn 1	0	7				
txn 2	0					

Buy on day 0, sell on day 1.
Profit = 7

prices = 8 $\leftrightarrow$ 15 5 70 80 100

profit table	day 0	day 1	day 2	day 3	day 4	day 5
txn 0	0	0	0	0	0	0
txn 1	0	7	7			
txn 2	0					

Buy on day 0, sell on day 1.
Profit = 7

prices = 8 15 5 $\leftrightarrow$ 70 80 100

profit table	day 0	day 1	day 2	day 3	day 4	day 5
txn 0	0	0	0	0	0	0
txn 1	0	7	7	65		
txn 2	0					

Buy on day 2, sell on day 3.
Profit = 65

prices = 8 15 5 $\leftarrow$ 70 $\rightarrow$ 80 100

profit table	day 0	day 1	day 2	day 3	day 4	day 5
txn 0	0	0	0	0	0	0
txn 1	0	7	7	65	75	
txn 2	0					

Buy on day 2, sell on day 4.
Profit = 75

prices = 8 15 5 $\leftarrow$ 70 80 $\rightarrow$ 100

profit table	day 0	day 1	day 2	day 3	day 4	day 5
txn 0	0	0	0	0	0	0
txn 1	0	7	7	65	75	95
txn 2	0					

Buy on day 2, sell on day 5.
Profit = 95



Profits with 2 Transactions

prices =	8	↔	15	5	70	80	100
profit table	day 0	day 1	day 2	day 3	day 4	day 5	
txn 0	0	0	0	0	0	0	
txn 1	0	7	7	65	75	95	
txn 2	0	7					

There are only 2 prices, hence can only complete 1 transaction.

Buy on day 0, sell on day 1.
Profit = 7

prices	8	↔	15	5	70	80	100
profit table	day 0	day 1	day 2	day 3	day 4	day 5	
txn 0	0	0	0	0	0	0	
txn 1	0	7	7	65	75	95	
txn 2	0	7	7				

There are only 3 prices, hence can only complete 1 transaction.

Buy on day 0, sell on day 1.
Profit = 7

prices	8 ↔ 15		5 ↔ 70		80	100
profit table	day 0	day 1	day 2	day 3	day 4	day 5
txn 0	0	0	0	0	0	0
txn 1	0	7	7	65	75	95
txn 2	0	7	7	72		

Buy on day 0, sell on day 1.
Profit = 7

Buy on day 2, sell on day 3.
Profit = 65

Total profit = 72

prices	8 ↔ 15 5 ← 70 → 80 100					
profit table	day 0	day 1	day 2	day 3	day 4	day 5
txn 0	0	0	0	0	0	0
txn 1	0	7	7	65	75	95
txn 2	0	7	7	72	82	

Buy on day 0, sell on day 1.
Profit = 7

Buy on day 2, sell on day 4.
Profit = 75

Total profit = 82

prices	8 ↔ 15 5 ← 70 80 → 100					
profit table	day 0	day 1	day 2	day 3	day 4	day 5
txn 0	0	0	0	0	0	0
txn 1	0	7		65	75	95
txn 2	0	7	7	72	82	102

Buy on day 0, sell on day 1.
Profit = 7

Buy on day 2, sell on day 5.
Profit = 95

Total profit = 102

A decorative image on the left side of the slide showing a stack of smooth, dark stones on a calm body of water, with their reflections visible. The stones are stacked horizontally, creating a sense of balance and tranquility.

Find Patterns

Scenario 1 — Not Making a Profit on The Day

If we're not completing a transaction and thus not making any profit on the day, then our profit would remain the same as the profit gained from the previous day.

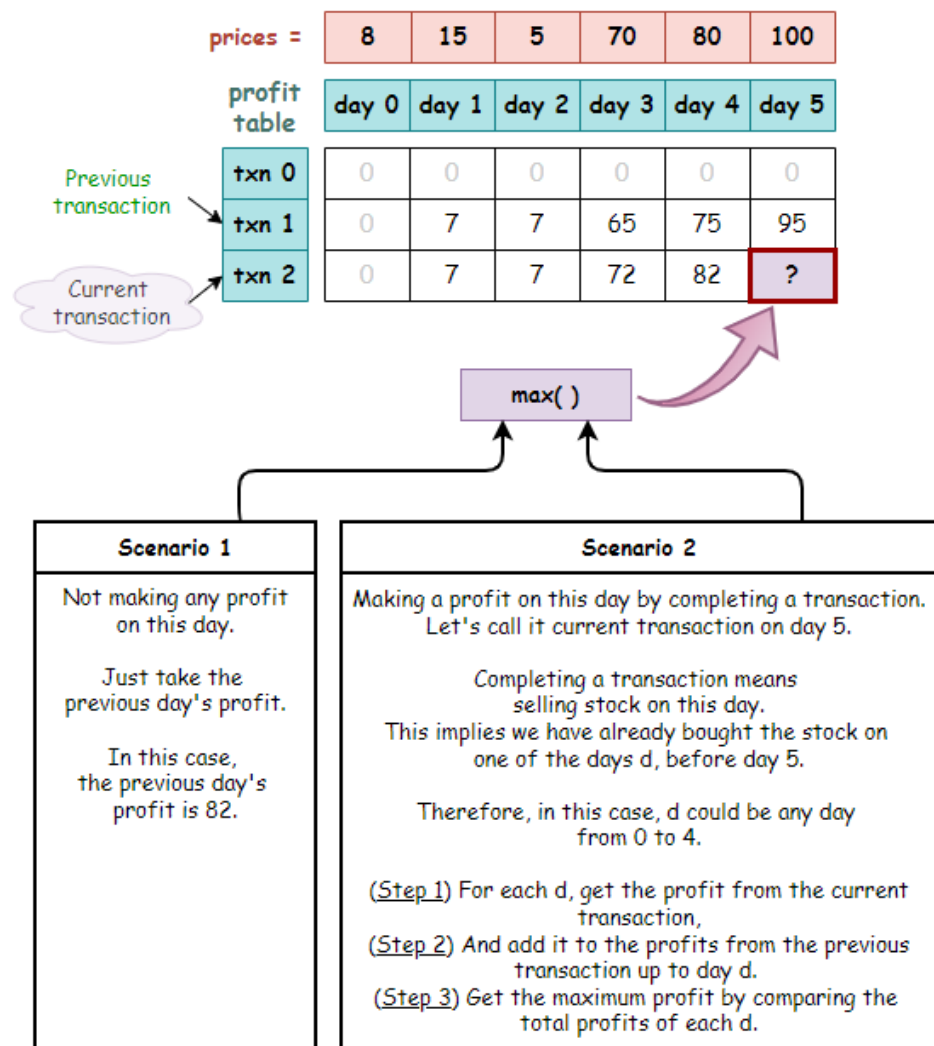
Scenario 2 — Making a Profit on The Day

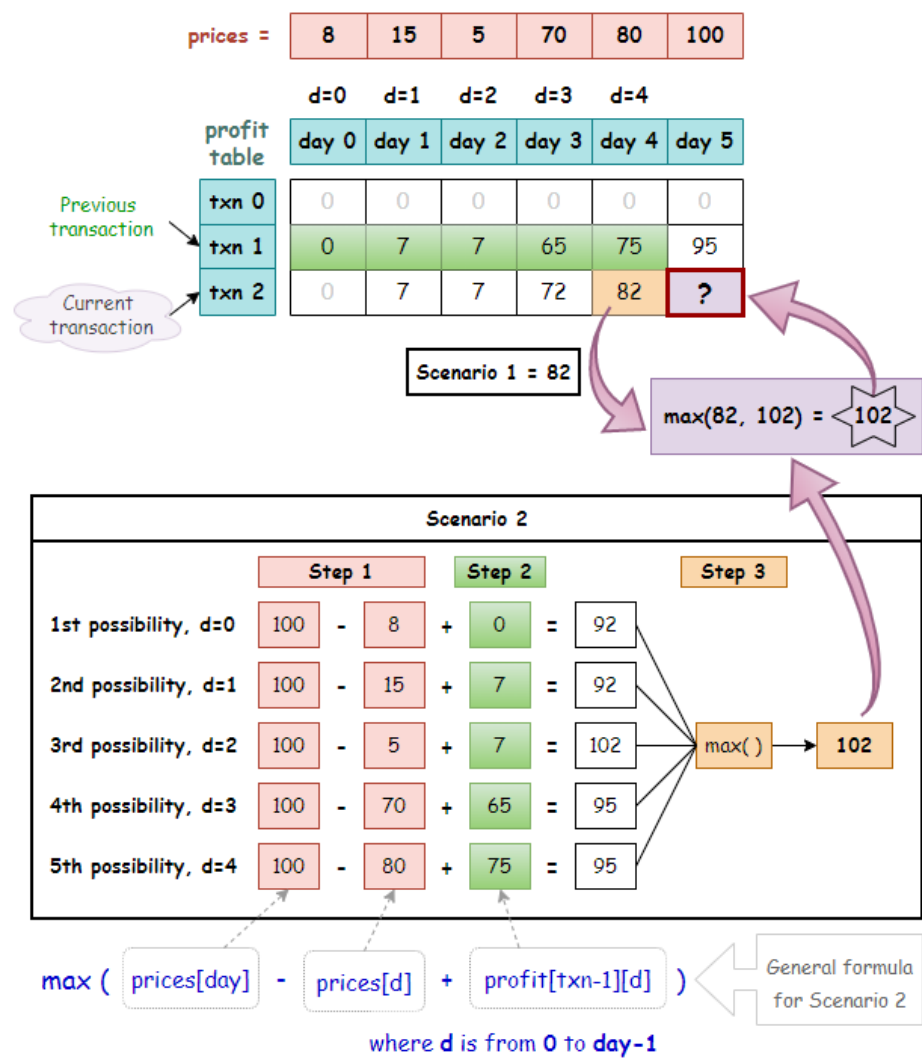
The second possibility is that we have already owned the stock and we want to sell it to make a profit on the day. That means we must have performed the buy action on one of the days before the current day. Let's refer to the day that the stock was bought as d . There are various possibilities of d . For example, if the current day is 5, then d could be any day from 0 to 4.

Note that the profit from selling the stock on the current day is just profit from one transaction. To get the total profit, we need to add this value to the profits gained from the previous transaction up to day d .

After obtaining the total profits for each possible d , we then apply the `max()` function to get the maximum value.

Solution Structure





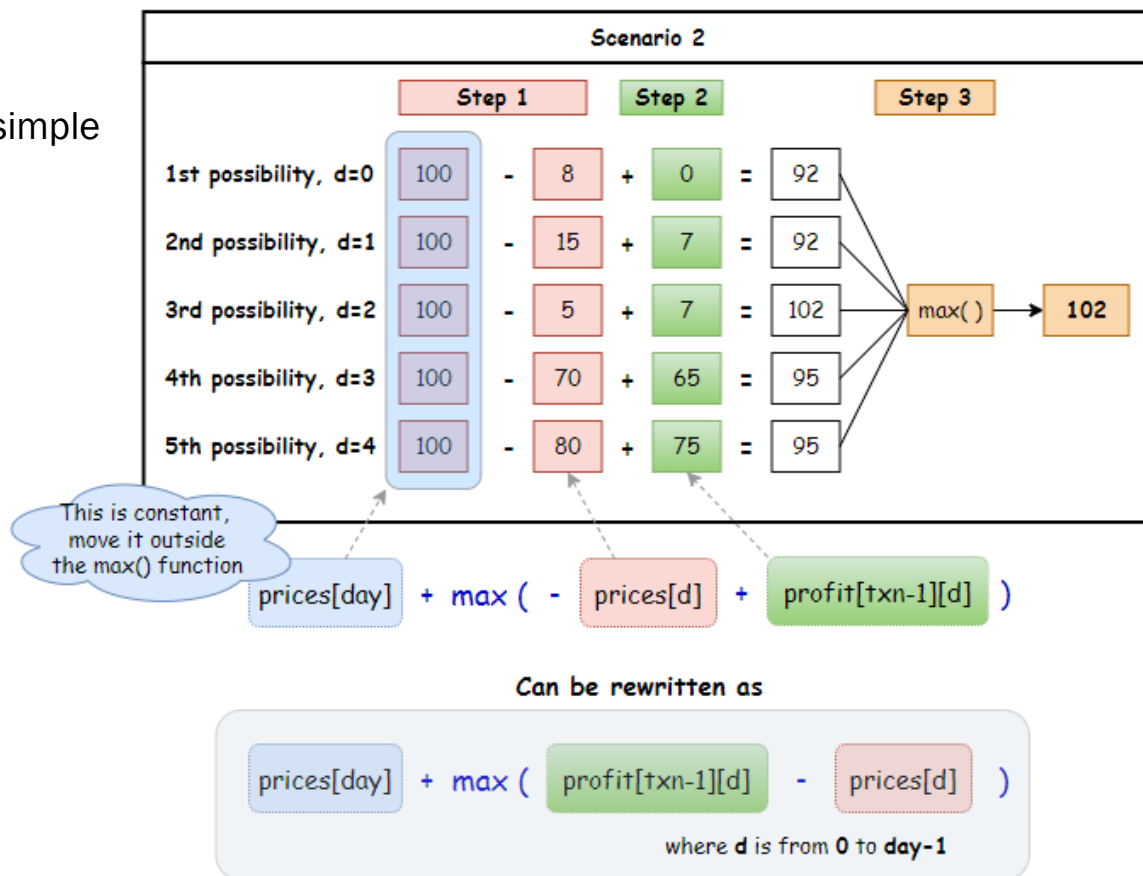


Complexity Analysis

The solution is okay. But can you do better?

Look For Ways to Improve

(i) Make it simple



(ii) Check for Redundancy

prices =

8	15	5	70	80	100
---	----	---	----	----	-----

	d=0	d=1	d=2	d=3	d=4	
profit table	day 0	day 1	day 2	day 3	day 4	day 5
txn 0	0	0	0	0	0	0
txn 1	0	7	7	65	75	95
txn 2	0	T1	T2	T3	T4	T5

Previous transaction → txn 1

Current transaction → txn 2

Calculation inside the max() function, i.e.
 $\text{profit}[\text{txn}-1][d] - \text{prices}[d]$

	T1	T2	T3	T4	T5
d = 0 :	0 - 8 = -8	0 - 8 = -8	0 - 8 = -8	0 - 8 = -8	0 - 8 = -8
d = 1 :		7 - 15 = -8	7 - 15 = -8	7 - 15 = -8	7 - 15 = -8
d = 2 :			7 - 5 = 2	7 - 5 = 2	7 - 5 = 2
d = 3 :				65 - 70 = -5	65 - 70 = -5
d = 4 :					75 - 80 = -5



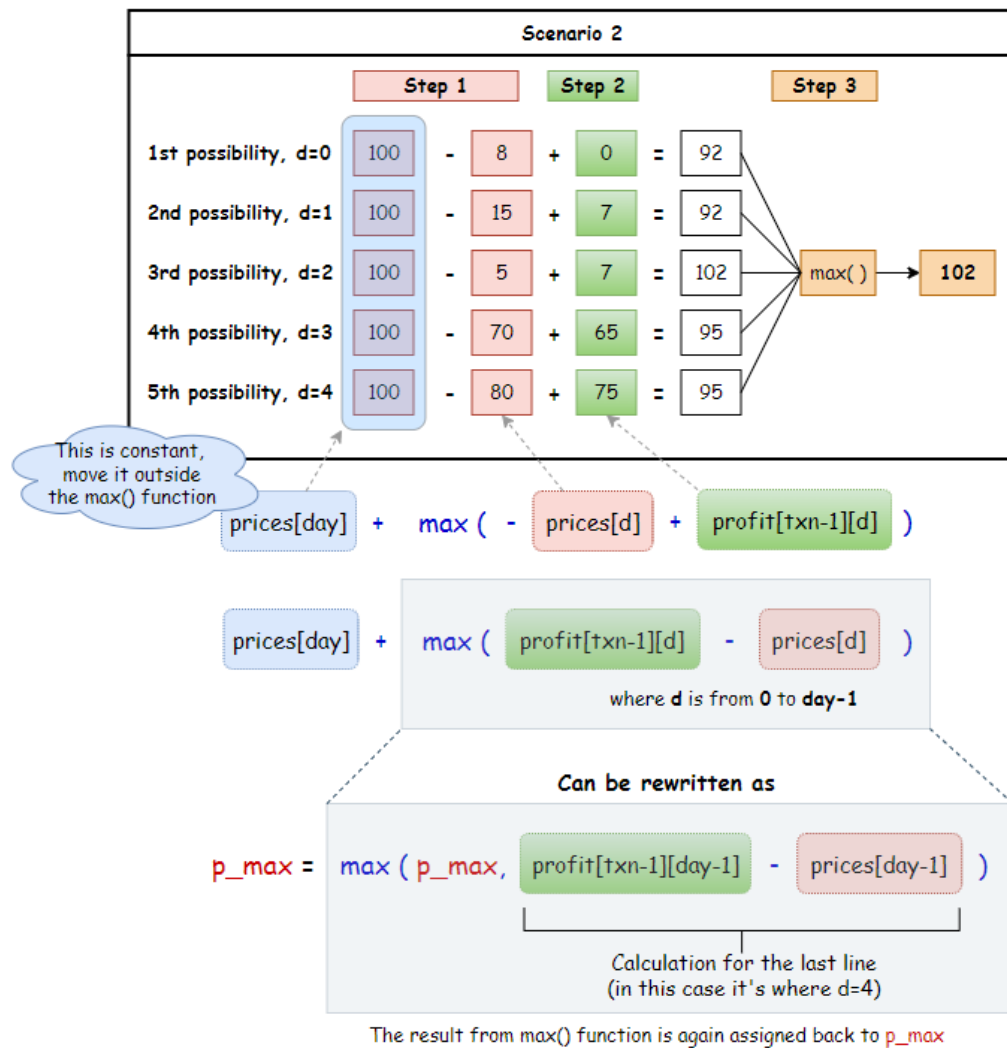
Repeated calculations

The first line is already calculated by T1

The first 2 lines are already calculated by T2

The first 3 lines are already calculated by T3

The first 4 lines are already calculated by T4





(iii) Determine the Scope of Dependency

$\text{curr_txn}[\text{day}] = \max(\text{curr_txn}[\text{day}-1] , \text{prices}[\text{day}] + \text{p_max})$

Computation for
current cell

Scenario 1

Scenario 2



Bottom-up Solution

```
def maxProfitBottomUp(k, prices):
    if len(prices) == 0 or k == 0:
        return 0
    even_array = [0 for _ in prices]
    odd_array = [0 for _ in prices]
    for txn in range(1, k+1): # Iterate through the transactions, starting from row 1.
        p_max = float("-inf") # Initialize p_max with negative infinity.
        if txn % 2 == 1:
            curr_txn = odd_array # We first start from row 1, so current transaction is on odd_array.
            prev_txn = even_array
        else:
            curr_txn = even_array
            prev_txn = odd_array
        for day in range(1, len(prices)): # Iterate through the prices of each day, starting from column 1.
            p_max = max(p_max, prev_txn[day-1] - prices[day-1])
            scenario1 = curr_txn[day-1]
            scenario2 = prices[day] + p_max
            curr_txn[day] = max(scenario1, scenario2)
    return even_array[-1] if k % 2 == 0 else odd_array[-1]
```



1. Optimal Substructure

Show that a solution to a problem consists of *making a choice*, which leaves one or more sub-problems to solve.

Suppose that you are given this last choice that leads to an optimal solution.

Given this choice, determine which sub-problems arise and how to characterize the resulting space of sub-problems.

Show that the solutions to the sub-problems used within the *optimal solution* must themselves be optimal. Usually use cut-and-paste.

Need to ensure that a wide enough range of choices and sub-problems are considered.



Optimal Substructure

Optimal substructure varies across problem domains:

1. How many sub-problems are used in an optimal solution.
2. How many choices in determining which sub-problem(s) to use.

Informally, running time depends on (# of sub-problems overall) x (# of choices).

How many sub-problems and choices do the examples considered contain?

Dynamic programming uses *optimal substructure bottom up*.

First find optimal solutions to sub-problems.

Then choose which to use in optimal solution to the problem.



Optimal Substructure

Does optimal substructure apply to all optimization problems? No.

Applies to determining the **shortest path** but **NOT** the **longest simple path** of an unweighted directed graph.

Why?

Shortest path has independent sub-problems.

Solution to one sub-problem does not affect solution to another sub-problem of the same problem.

Sub-problems are not independent in longest simple path.

Solution to one sub-problem affects the solutions to other sub-problems.

A decorative image on the left side of the slide showing a stack of smooth, dark stones on a calm body of water, with their reflections visible. The stones are stacked in a slightly offset manner, creating a sense of balance and tranquility.

2. Overlapping Subproblems

The space of sub-problems must be “small”.

The total number of distinct sub-problems is a polynomial in the input size.

A recursive algorithm is exponential because it solves the same problems repeatedly.

If divide-and-conquer is applicable, then each problem solved will be brand new.



DP & DC

Like **divide and conquer**, DP solves problems by combining solutions to sub-problems.

Unlike divide and conquer, sub-problems are not independent.

- Sub-problems may share sub-sub-problems,

- However, solution to one sub-problem may not affect the solutions to other sub-problems of the same problem. (More on this later.)

DP reduces computation by

- Solving sub-problems in a bottom-up fashion.

- Storing solution to a sub-problem the first time it is solved.

- Looking up the solution when sub-problem is encountered again.

Key: determine structure of optimal solutions



Steps in Dynamic Programming

1. Characterize structure of an optimal solution.
2. Define value of optimal solution *recursively*.
3. Compute optimal solution values either top-down with caching or *bottom-up in a table*.
4. Construct an optimal solution from computed values.

Longest Common Subsequence

Problem: Given 2 sequences, $X = \{x_1, \dots, x_m\}$ and $Y = \{y_1, \dots, y_n\}$, find a common subsequence whose length is maximum.

Subsequence **need not be consecutive**, but **must be in order**.



For every subsequence of X , check whether it's a subsequence of Y .

Time: $\Theta(n2^m)$.

2^m subsequences of X to check.

Each subsequence takes $\Theta(n)$ time to check: scan Y for first letter, for second, and so on.

Longest Common Subsequence

Optimal Substructure

Theorem

Let $Z = \langle z_1, \dots, z_k \rangle$ be any LCS of X and Y .

1. If $x_m = y_n$, then $z_k = x_m = y_n$ and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1} .
2. If $x_m \neq y_n$, then either $z_k \neq x_m$ and Z is an LCS of X_{m-1} and Y .
3. or $z_k \neq y_n$ and Z is an LCS of X and Y_{n-1} .

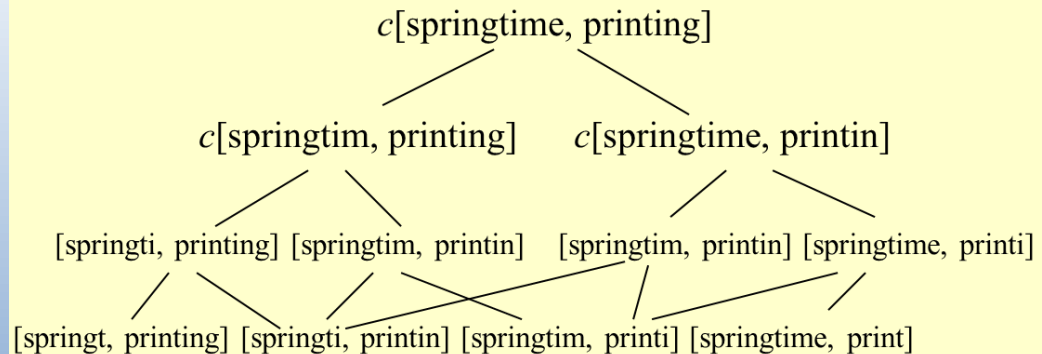
Recursive Solution

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max(c[i-1, j], c[i, j-1]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases}$$

Where $c[i, j]$ = length of LCS of X_i and Y_j

Longest Common Subsequence

$$c[\alpha, \beta] = \begin{cases} 0 & \text{if } \alpha \text{ empty or } \beta \text{ empty,} \\ c[\text{prefix}\alpha, \text{prefix}\beta] + 1 & \text{if } \text{end}(\alpha) = \text{end}(\beta), \\ \max(c[\text{prefix}\alpha, \beta], c[\alpha, \text{prefix}\beta]) & \text{if } \text{end}(\alpha) \neq \text{end}(\beta). \end{cases}$$





Longest Common Subsequence

$$c[\alpha, \beta] = \begin{cases} 0 & \text{if } \alpha \text{ empty or } \beta \text{ empty,} \\ c[\text{prefix}\alpha, \text{prefix}\beta] + 1 & \text{if } \text{end}(\alpha) = \text{end}(\beta), \\ \max(c[\text{prefix}\alpha, \beta], c[\alpha, \text{prefix}\beta]) & \text{if } \text{end}(\alpha) \neq \text{end}(\beta). \end{cases}$$

Keep track of $c[\alpha, \beta]$ in a table of nm entries:

- top/down
- bottom/up

Example

		0	1	2	3	4	5	6	7
			<i>M</i>	<i>Z</i>	<i>J</i>	<i>A</i>	<i>W</i>	<i>X</i>	<i>U</i>
0		<u>0</u>	0	0	0	0	0	0	0
1	<i>X</i>	<u>0</u>	0	0	0	0	0	1	1
2	<i>M</i>	0	<u>1</u>	<u>1</u>	1	1	1	1	1
3	<i>J</i>	0	1	1	<u>2</u>	2	2	2	2
4	<i>Y</i>	0	1	1	<u>2</u>	2	2	2	2
5	<i>A</i>	0	1	1	2	<u>3</u>	<u>3</u>	<u>3</u>	3
6	<i>U</i>	0	1	1	2	3	3	3	<u>4</u>
7	<i>Z</i>	0	1	2	2	3	3	3	<u>4</u>

Longest Common Subsequence

LCS-LENGTH (X, Y)

```
1.   $m \leftarrow \text{length}[X]$ 
2.   $n \leftarrow \text{length}[Y]$ 
3.  for  $i \leftarrow 1$  to  $m$ 
4.      do  $c[i, 0] \leftarrow 0$ 
5.  for  $j \leftarrow 0$  to  $n$ 
6.      do  $c[0, j] \leftarrow 0$ 
7.  for  $i \leftarrow 1$  to  $m$ 
8.      do for  $j \leftarrow 1$  to  $n$ 
9.          do if  $x_i = y_j$ 
10.             then  $c[i, j] \leftarrow c[i-1, j-1] + 1$ 
11.                  $b[i, j] \leftarrow \text{" "}$ 
12.             else if  $c[i-1, j] \geq c[i, j-1]$ 
13.                 then  $c[i, j] \leftarrow c[i-1, j]$ 
14.                      $b[i, j] \leftarrow \text{"\u2191"}$ 
15.             else  $c[i, j] \leftarrow c[i, j-1]$ 
16.                  $b[i, j] \leftarrow \text{"\u2190"}$ 
17.  return  $c$  and  $b$ 
```

$b[i, j]$ points to table entry whose subproblem we used in solving LCS of X_i and Y_j .

$c[m, n]$ contains the length of an LCS of X and Y .

Time: $O(mn)$



Longest Common Subsequence

PRINT-LCS (b, X, i, j)

1. **if** $i = 0$ or $j = 0$
2. **then return**
3. **if** $b[i, j] = \text{“ ”}$
4. **then** PRINT-LCS($b, X, i-1, j-1$)
5. print x_i
6. **elseif** $b[i, j] = \text{“}\uparrow\text{”}$
7. **then** PRINT-LCS($b, X, i-1, j$)
8. **else** PRINT-LCS($b, X, i, j-1$)

- Initial call is PRINT-LCS (b, X, m, n).
- When $b[i, j] = \text{“ ”}$, we have extended LCS by one character.
So LCS = entries with in them.
- Time: $O(m+n)$



Matrix Chain Multiplication (MCM)

Input: Matrices $A_1, A_2, \dots, A_n$, each A_i of size $P_{i-1} \times P_i$

Output: Fully parenthesized product $A_1 A_2 \dots A_n$ that minimizes the number of scalar multiplications

A product of matrices is fully parenthesized if it is either

1. a single matrix
2. the product of 2 fully parenthesized matrix products surrounded by parentheses

Example of MCM problem:

We have 3 matrices and the size of each matrix:

A_1 (10 x 100), A_2 (100 x 5), A_3 (5 x 50)

We can fully parenthesized them in two ways:

1. $(A_1 (A_2 A_3)) = 100 \times 5 \times 50 + 10 \times 100 \times 50 = 75000$
2. $((A_1 A_2) A_3) = 10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$ (*10 times better*)



Matrix Chain Multiplication (MCM)

Step 1: characterize the optimal sub-structure of this problem.

Let $A_{i..j}$ ($i < j$) denote the result of multiplying $A_i A_{i+1} \dots A_j$.

$A_{i..j}$ can be obtained by splitting it into $A_{i..k}$ and $A_{k+1..j}$ and then multiplying the sub-products. There are $j-i$ possible splits (i.e. $k=i, \dots, j-1$)

Within the optimal parenthesization of $A_{i..j}$:

- (a) the parenthesization of $A_{i..k}$ must be optimal
- (b) the parenthesization of $A_{k+1..j}$ must be optimal

Because if they are not optimal, then there exist other split which is better, and we should choose that split and not this split.



Matrix Chain Multiplication (MCM)

Step 2: Recursive formulation

Need to find $A_{1..n}$

Let $m[i,j]$ = minimum number of scalar multiplications needed to compute $A_{i..j}$

Since $A_{i..j}$ can be obtained by breaking it into $A_{i..k} A_{k+1..j}$, we have

- $m[i,j] = 0$, if $i=j$

- $m[i,j] = \min_{i \leq k < j} \{ m[i,k] + m[k+1,j] + p_{i-1} p_k p_j \}$, if $i < j$

let $s[i,j]$ be the value k where the optimal split occurs.



$A_1 : 30 \times 35$, $A_2 : 35 \times 15$, $A_3 : 15 \times 5$, $A_4 : 5 \times 10$, $A_5 : 10 \times 20$, $A_6 : 20 \times 25$

	i=1	2	3	4	5	6
J=6						
5						
4						
3						
2						
1						

$m[1..6, 1..6]$

	i=1	2	3	4	5	6
J=6						
5						
4						
3						
2						
1						

$s[1..6, 1..6]$

$$m[2, 5] = \min \begin{cases} m[2, 2] + m[3, 5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20 = 13000, \\ m[2, 3] + m[4, 5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20 = 7125, \\ m[2, 4] + m[5, 5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20 = 11375, \end{cases} = 7125$$

Matrix Chain Multiplication (MCM)

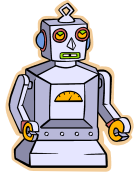
Step 3 Computing the Optimal Costs

```
MatrixChainMultiply ( $\langle A_1, \dots, A_n \rangle$ ,  $s[1..n, 1..n]$ ,  $i$ ,  $j$ ) {  
  if ( $i < j$ ) {  
     $A_{l..s[i,j]} =$   
    MatrixChainMultiply ( $\langle A_1, \dots, A_n \rangle$ ,  $s[1..n, 1..n]$ ,  $i$ ,  $s[i, j]$ );  
     $A_{s[i,j]+1..j} =$   
    MatrixChainMultiply ( $\langle A_1, \dots, A_n \rangle$ ,  $s[1..n, 1..n]$ ,  $s[i, j] + 1$ ,  $j$ );  
    return  $A_{l..s[i,j]} A_{s[i,j]+1..j}$ ;  
  }  
  else { return  $A_i$  }  
}
```

Step 4: Constructing an Optimal Solution

```
Print-MCM( $s, i, j$ )  
  if  $i = j$  then  
    print  $A_i$   
  else  
    print "(" + Print-MCM( $s, 1, s[i, j]$ ) + "*" + Print-MCM( $s, s[i, j] + 1, j$ ) + ")"
```

Knapsack Problem



Get as much value
as you can
into the knapsack



The (General) 0-1 Knapsack Problem

0-1 knapsack problem:

- n items.
- Item i is worth $\$v_i$, weighs w_i pounds.
- Find a most valuable subset of items with total weight $\leq W$.
- v_i , w_i and W are all integers.
- Have to either take an item or not take it - can't take part of it.



Dynamic Programming Solution

The General 0-1 Knapsack Problem can be solved by dynamic programming.

Let W = capacity of knapsack (kg)

Let (v_i, w_i) = value (\$) and weight (kg) of item $i \in [1..n]$

Let $c[i, w]$ = value of optimal solution for knapsack of capacity w and items drawn from $[1..i]$

$$\text{Then } c[i, w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ c[i-1, w] & \text{if } w_i > w \\ \max(v_i + c[i-1, w-w_i], c[i-1, w]) & \text{if } i > 0 \text{ and } w_i \leq w \end{cases}$$

A vertical image showing four smooth, dark, rounded stones stacked on a calm body of water. The stones are dark and smooth, and their reflection is visible in the water below. The background is a light, hazy sky.

Let $c[i, w]$ = value of optimal solution for knapsack of capacity w and items drawn from $[1...i]$

62

Bottom-Up Computation

Let W = capacity of knapsack (kg)

Let (v_i, w_i) = value (\$) and weight (kg) of item $i \in [1..n]$

Let $c[i, w]$ = value of optimal solution for knapsack of capacity w and items drawn from $[1..i]$

$$\text{Then } c[i, w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ c[i-1, w] & \text{if } w_i > w \\ \max(v_i + c[i-1, w-w_i], c[i-1, w]) & \text{if } i > 0 \text{ and } w \geq w_i \end{cases}$$

Need only ensure that $c[i-1, v]$ has been computed $\forall v \leq w$

$c[i, w]$		w							
i	Allowed Items	0	1	2	...	w	...	W	
0	{}								
1	{1}								
2	{1 2}								
...	...								
i	{1 2 ... i }					$c[i, w]$			
...	...								
n	{1 2 ... n }								



Integer Knapsack Solution

DYNAMIC-0-1-KNAPSACK(v, w, n, W)

```
for  $w \leftarrow 0$  to  $W$ 
  do  $c[0, w] \leftarrow 0$ 
for  $i \leftarrow 1$  to  $n$ 
  do  $c[i, 0] \leftarrow 0$ 
    for  $w \leftarrow 1$  to  $W$ 
      do if  $w_i \leq w$ 
        then if  $v_i + c[i - 1, w - w_i] > c[i - 1, w]$ 
          then  $c[i, w] \leftarrow v_i + c[i - 1, w - w_i]$ 
          else  $c[i, w] \leftarrow c[i - 1, w]$ 
        else  $c[i, w] \leftarrow c[i - 1, w]$ 
```

Value=0 if no items or no knapsack.

Integer Knapsack Solution

DYNAMIC-0-1-KNAPSACK(v, w, n, W)

for $w \leftarrow 0$ **to** W

do $c[0, w] \leftarrow 0$

for $i \leftarrow 1$ **to** n

do $c[i, 0] \leftarrow 0$

for $w \leftarrow 1$ **to** W

do if $w_i \leq w$

then if $v_i + c[i - 1, w - w_i] > c[i - 1, w]$

then $c[i, w] \leftarrow v_i + c[i - 1, w - w_i]$

else $c[i, w] \leftarrow c[i - 1, w]$

else $c[i, w] \leftarrow c[i - 1, w]$

Fill in table row-wise

Recurrence relation

Example

Capacity $W = 6$

$$\text{Then } c[i, w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ c[i-1, w] & \text{if } w_i > w \\ \max(v_i + c[i-1, w - w_i], c[i-1, w]) & \text{if } i > 0 \text{ and } w \geq w_i \end{cases}$$

i	v	w
1	1	2
2	3	3
3	5	1
4	2	5
5	6	3
6	10	5

$c[i, w]$		w						
i	Allowed Items	0	1	2	3	4	5	6
0	$\{\}$	0	0	0	0	0	0	0
1	$\{1\}$	0	0	1	1	1	1	1
2	$\{1\ 2\}$	0	0	1	3	3	4	4
3	$\{1\ 2\ 3\}$	0	5	5	6	8	8	9
4	$\{1\ 2\ 3\ 4\}$	0	5	5	6	8	8	9
5	$\{1\ 2\ 3\ 4\ 5\}$	0	5	5	6	11	11	12
6	$\{1\ 2\ 3\ 4\ 5\ 6\}$	0	5	5	6	11	11	15

Solving for the Items to Pack

$$\text{Then } c[i, w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ c[i-1, w] & \text{if } w_i > w \\ \max(v_i + c[i-1, w - w_i], c[i-1, w]) & \text{if } i > 0 \text{ and } w \geq w_i \end{cases}$$

i	v	w
1	1	2
2	3	3
3	5	1
4	2	5
5	6	3
6	10	5

$c[i, w]$		w							
i	Allowed Items	0	1	2	3	4	5	6	
0	{}	0	0	0	0	0	0	0	
1	{1}	0	0	1	1	1	1	1	
2	{1 2}	0	0	1	3	3	4	4	
3	{1 2 3}	0	5	5	6	8	8	9	
4	{1 2 3 4}	0	5	5	6	8	8	9	
5	{1 2 3 4 5}	0	5	5	6	11	11	12	
6	{1 2 3 4 5 6}	0	5	5	6	11	11	15	

$i = n$

$w = W$

items = {}

loop for $i = n$ downto 1

if $c[i, w] > c[i-1, w]$

items = items + { i }

$w = w - w_i$

Rock Climbing Problem

A rock climber wants to get from the bottom of a rock to the top by the safest possible path.

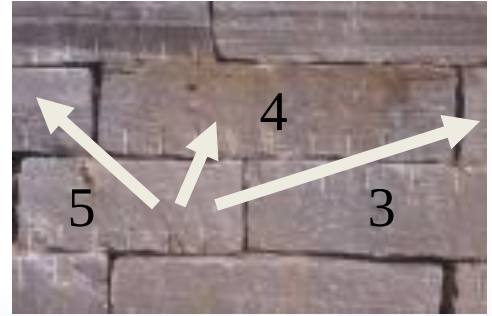
At every step, he reaches for handholds above him; some holds are safer than other.

From every place, he can only reach a few nearest handholds.



Rock climbing

Suppose we have a wall instead of the rock.



At every step our climber can reach exactly three handholds: above, above and to the right and above and to the left.

There is a table of “danger ratings” provided. The “danger” of a path is the sum of danger ratings of all handholds on the path.

Rock Climbing

- We represent the wall as a table.
- Every cell of the table contains the danger rating of the corresponding block.

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

The obvious greedy algorithm does not give an optimal solution. The rating of **this path** is 13.

The rating of an **optimal path** is 12.

However, we can solve this problem by a dynamic programming strategy in polynomial time.



Rock climbing: step 1.

Step 1: Describe an array of values you want to compute.

For $1 \leq i \leq n$ and $1 \leq j \leq m$, define $A(i,j)$ to be the cumulative rating of the least dangerous path from the bottom to the hold (i,j) .

The rating of the best path to the top will be the minimal value in the last row of the array.



Rock climbing: step 2.

Step 2: Give a recurrence for computing later values from earlier (bottom-up).

Let $C(i,j)$ be the rating of the hold (i,j) . There are three cases for $A(i,j)$:

Left ($j=1$): $C(i,j) + \min\{A(i-1,j), A(i-1,j+1)\}$

Right ($j=m$): $C(i,j) + \min\{A(i-1,j-1), A(i-1,j)\}$

Middle: $C(i,j) + \min\{A(i-1,j-1), A(i-1,j), A(i-1,j+1)\}$

For the first row ($i=1$), $A(i,j) = C(i,j)$.



Rock climbing: simpler step 2

Add initialization row: $A(0,j)=0$. No danger to stand on the ground.

Add two initialization columns: $A(i,0)=A(i,m+1)=\infty$. It is infinitely dangerous to try to hold on to the air where the wall ends.

Now the recurrence becomes, for every i,j :

$$A(i,j) = C(i,j) + \min\{A(i-1,j-1), A(i-1,j), A(i-1,j+1)\}$$

Rock climbing: example

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \setminus j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞						∞
2	∞						∞
3	∞						∞
4	∞						∞

Initialization: $A(i,0)=A(i,m+1)=\infty$, $A(0,j)=0$



Rock climbing: example

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \setminus j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞						∞
3	∞						∞
4	∞						∞

The values in the first row are the same as $C(i,j)$.

Rock climbing: example

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \setminus j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7					∞
3	∞						∞
4	∞						∞

$$A(2,1)=5+\min\{\infty,3,2\}=7.$$

Rock climbing: example

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \setminus j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7	9				∞
3	∞						∞
4	∞						∞

$$A(2,1)=5+\min\{\infty,3,2\}=7. \quad A(2,2)=7+\min\{3,2,5\}=9$$

Rock climbing: example

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \setminus j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7	9	7			∞
3	∞						∞
4	∞						∞

$A(2,1)=5+\min\{\infty,3,2\}=7$. $A(2,2)=7+\min\{3,2,5\}=9$
 $A(2,3)=5+\min\{2,5,4\}=7$.

Rock climbing: example

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \backslash j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7	9	7	10	5	∞
3	∞						∞
4	∞						∞

The best cumulative rating on the second row is 5.

Rock climbing: example

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \backslash j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7	9	7	10	5	∞
3	∞	11	11	13	7	8	∞
4	∞						∞

The best cumulative rating on the third row is 7.

Rock climbing: example

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

i\j	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7	9	7	10	5	∞
3	∞	11	11	13	7	8	∞
4	∞	13	19	16	12	15	∞

The best cumulative rating on the last row is 12.



Rock climbing: example

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \backslash j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7	9	7	10	5	∞
3	∞	11	11	13	7	8	∞
4	∞	13	19	16	12	15	∞

The best cumulative rating on the last row is 12.

So the rating of the best path to the top is 12.

Rock climbing example: step 4

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

i\j	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7	9	7	10	5	∞
3	∞	11	11	13	7	8	∞
4	∞	13	19	16	12	15	∞

To find the actual path we need to retrace backwards the decisions made during the calculation of $A(i,j)$.

Rock climbing example: step 4

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \setminus j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7	9	7	10	5	∞
3	∞	11	11	13	7	8	∞
4	∞	13	19	16	12	15	∞

The last hold was (4,4).

To find the actual path we need to retrace backwards the decisions made during the calculation of $A(i,j)$.

Rock climbing example: step 4

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \setminus j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7	9	7	10	5	∞
3	∞	11	11	13	7	8	∞
4	∞	13	19	16	12	15	∞

The hold before the last was (3,4), since $\min\{13,7,8\}$ was 7.

To find the actual path we need to retrace backwards the decisions made during the calculation of $A(i,j)$.

Rock climbing example: step 4

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \setminus j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7	9	7	10	5	∞
3	∞	11	11	13	7	8	∞
4	∞	13	19	16	12	15	∞

The hold before that was (2,5), since $\min\{7,10,5\}$ was 5.

To find the actual path we need to retrace backwards the decisions made during the calculation of $A(i,j)$.

Rock climbing example: step 4

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \setminus j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7	9	7	10	5	∞
3	∞	11	11	13	7	8	∞
4	∞	13	19	16	12	15	∞

Finally, the first hold was (1,4), since $\min\{4,8\}$ was 4.

To find the actual path we need to retrace backwards the decisions made during the calculation of $A(i,j)$.

Rock climbing example: step 4

$C(i,j)$:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

$A(i,j)$:

$i \setminus j$	0	1	2	3	4	5	6
0	∞	0	0	0	0	0	∞
1	∞	3	2	5	4	8	∞
2	∞	7	9	7	10	5	∞
3	∞	11	11	13	7	8	∞
4	∞	13	19	16	12	15	∞

We are done!

